PES-VCS Lab Report

Sai Harshith <PES1UG24CS398>

Repository implementation for the OS Unit 4 Orange Problem: content-addressable object storage, recursive tree objects, staging index, commits, history, references, and analysis questions.

Phase	Implemented
1	object_write, object_read, hashing, sharding, atomic writes, integrity verification
2	tree serialization usage and recursive tree_from_index
3	index_load, index_save, index_add with metadata and atomic writes
4	commit_create with tree snapshots, parent links, author metadata, and HEAD update

Required Screenshots

Screenshot 1A: ./test_objects output showing all Phase 1 tests passing

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 1B: find .pes/objects -type f showing sharded object storage

```
All Phase 1 tests passed.
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 2A: ./test_tree output showing all Phase 2 tests passing

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 2B: xxd of a raw tree object showing the binary tree format

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ for f in $(find .pes/objects -type f); do
  if head -c 5 "$f" | grep -q "tree "; then
    echo "$f"
    xxd "$f" | head -20
    break
  fi
done
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
00000000: 7472 6565 2030 00                        tree 0.
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 3A: pes init, pes add, pes status sequence

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ rm -f file1.txt file2.txt core
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ echo "hello" > file1.txt
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ echo "world" > file2.txt
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ ./pes add file1.txt file2.txt
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  report.pdf
  untracked:  win_compat.h
  untracked:  pes.h
  untracked:  pes.c
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  object.c
  untracked:  tree.h
  untracked:  test_objects
  untracked:  REPORT.md
  untracked:  commit.h
  untracked:  .gitattributes
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  index.c
  untracked:  test_tree
```

Screenshot 3B: cat .pes/index showing the text-format index

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776693858 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776693864 6 file2.txt
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 4A: pes log output showing three commits

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ rm -rf .pes
rm -f hello.txt bye.txt
./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 06a4f1be94b4... Initial commit
Committed: 68b105ea04f4... Add world
Committed: acb45b9d2ff4... Add farewell
commit acb45b9d2ff474dc6d45fb5be5ed2a7f39cfe1791a7c3be5f70f68836a9f0bbd
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039

    Add farewell

commit 68b105ea04f4f0b6ebe1d48bb789a2647993b5a2b7ffdfe152466f09d80ae728
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039

    Add world

commit 06a4f1be94b4356b089e4023dca6aca50a4c2e19299d33f243915bbac5858a62
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039
```


```
./pes log
Initialized empty PES repository in .pes/
Committed: 06a4f1be94b4... Initial commit
Committed: 68b105ea04f4... Add world
Committed: acb45b9d2ff4... Add farewell
commit acb45b9d2ff474dc6d45fb5be5ed2a7f39cfe1791a7c3be5f70f68836a9f0bbd
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039

    Add farewell

commit 68b105ea04f4f0b6ebe1d48bb789a2647993b5a2b7ffdfe152466f09d80ae728
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039

    Add world

commit 06a4f1be94b4356b089e4023dca6aca50a4c2e19299d33f243915bbac5858a62
Author: Sai Harshith <PES1UG24CS398>
Date:   1776694039

    Initial commit
```


Screenshot 4B: find .pes -type f | sort after three commits

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/06/a4f1be94b4356b089e4023dca6aca50a4c2e19299d33f243915bbac5858a62
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/68/b105ea04f4f0b6ebe1d48bb789a2647993b5a2b7ffdfe152466f09d80ae728
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/ac/b45b9d2ff474dc6d45fb5be5ed2a7f39cfe1791a7c3be5f70f68836a9f0bbd
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ cat .pes/refs/heads/main
acb45b9d2ff474dc6d45fb5be5ed2a7f39cfe1791a7c3be5f70f68836a9f0bbd
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ s
```

Final Integration: Full integration test output

```
harshith@harshith-VirtualBox:~/PES1UG24CS398-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: c6b351a8f710... Initial commit


Log after first commit:
commit c6b351a8f7109030a3a77f18d6bbb34070e5c35d5ac73a8a53c6f495fa6941eb
Author: Sai Harshith <PES1UG24CS398>
Date:   1776695175

    Initial commit


--- Second Commit ---
Committed: 7a190004824d... Update file.txt

--- Third Commit ---
Committed: 1a5c34664767... Add farewell

--- Full History ---
commit 1a5c3466476776020a162cb3937f90ab6ef789b2909aefd9a5b3db3552f53690
Author: Sai Harshith <PES1UG24CS398>
Date:   1776695175

    Add farewell

commit 7a190004824df8b99a0de9ef05610603f56c2c52be52c986a8de64332990a775
Author: Sai Harshith <PES1UG24CS398>
Date:   1776695175

    Update file.txt

commit c6b351a8f7109030a3a77f18d6bbb34070e5c35d5ac73a8a53c6f495fa6941eb
Author: Sai Harshith <PES1UG24CS398>
Date:   1776695175

    Initial commit
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
1a5c3466476776020a162cb3937f90ab6ef789b2909aefd9a5b3db3552f53690

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1a/5c3466476776020a162cb3937f90ab6ef789b2909aefd9a5b3db3552f53690
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7a/190004824df8b99a0de9ef05610603f56c2c52be52c986a8de64332990a775
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c6/b351a8f7109030a3a77f18d6bbb34070e5c35d5ac73a8a53c6f495fa6941eb
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5: Branching and Checkout

Q5.1

To implement pes checkout , PES would first verify that .pes/refs/heads/ exists, read the commit hash stored there, update .pes/HEAD to contain ref: refs/heads/, then replace the working directory contents with the tree pointed to by that commit. That means reading the commit object, walking its root tree and subtrees, materializing blob contents as files, creating directories, setting modes, and updating the index to match the checked-out snapshot. The complexity is that checkout is not just changing one ref file: it must safely transform live user files without deleting uncommitted work, must handle files that become directories or directories that become files, and must leave the index, HEAD, and working directory consistent if an error happens halfway.

Q5.2

A dirty checkout conflict can be detected by comparing the working directory against the index before replacing files. For each tracked index entry, stat the path and compare size and mtime to the index metadata; for higher confidence, re-hash the working file as a blob and compare the blob hash to the index hash. Then read the target branch commit tree and check whether that path would be different in the target snapshot. If the working file differs from the index and the target branch also wants to write a different blob at that path, checkout must refuse because it would overwrite uncommitted work. Deleted tracked files and type changes should be treated the same way.

Q5.3

Detached HEAD means .pes/HEAD stores a commit hash directly instead of a symbolic ref such as ref: refs/heads/main. New commits made in this state can still be valid: each new commit points to the detached commit as its parent, and HEAD moves to the new hash. The risk is that no branch name points to those commits, so switching away can make them unreachable by normal branch traversal. A user could recover them by creating a branch at the detached commit hash before leaving, by using the printed commit hash from pes commit or pes log, or, in a fuller system, by consulting a reflog and writing that hash into .pes/refs/heads/recovered.

Q6: Garbage Collection and Space Reclamation

Q6.1

A mark-and-sweep collector would first enumerate all branch refs in .pes/refs/heads and push their commit hashes onto a stack or queue. It would maintain a hash set of reachable object IDs. For each commit popped, mark it, parse it, enqueue its parent commit if present, and enqueue its root tree. For each tree, mark it, parse every entry, enqueue child trees, and mark blob hashes. After marking, walk .pes/objects and delete every object hash not present in the reachable set. A hash set gives O(1) average membership checks. With 100,000 commits and 50 branches, the commit visit count is at most 100,000 if histories share ancestry and the set avoids revisits. The total object visits are commits plus all distinct reachable trees and blobs; in a snapshot-based repository this may be many more than 100,000, but each unique object is still visited once.

Q6.2

Concurrent garbage collection is dangerous because commit creation publishes objects in several steps. A process might write new blob and tree objects, but not yet write or update the final commit/ref. A concurrent GC that starts from current branch refs cannot see those new objects, classifies them as unreachable, and deletes one just before the commit object starts referencing it. The result is a commit whose tree or blob is missing. Real Git reduces this risk with conservative pruning rules, lock files, temp files, grace periods for recent unreachable objects, pack/ref locking coordination, and by avoiding deletion of objects that may have been created recently by another process.

Verification Notes

Local verification was performed on Windows with MinGW. The assignment target is Ubuntu 22.04; for local Windows-only execution I used the OpenSSL include/lib paths, a small win_compat.h include, and a larger Windows linker stack because the provided CLI allocates a large Index on the stack. The source remains compatible with the POSIX workflow described in the guide.